A Model-Driven Methodology for Designing Data-Intensive Architectures

Anonymous Authors

***Abstract*—Designing data-intensive software architectures is a challenging task that often relies on manual, unstructured processes, leading to design decisions that are implicit and poorly documented. This paper proposes a systematic, model-driven methodology to semi-automate the design of data-intensive architectures. We introduce a formal Scenario Description Language (SDL) to capture application requirements, focusing on data characteristics and consumer needs, and an Architecture Description Language (ADL) to model architectural components. Our methodology automatically translates an SDL into suitable candidate architectures and guides the selection of concrete technologies by mapping architectural components to existing data management and processing systems.**

We evaluate our approach through complementary methods: (i) we apply the methodology to two real-world industrial use cases to assess its correctness in suggesting suitable software architectures for the scenarios at hand; (ii) we conduct a survey with academic and industrial experts to evaluate the quality and usability of our SDL; (iii) we adopt these use cases to evaluate the completeness of our ADL.

Results show that the methodology generates architectures that are aligned with expert architects, while making the design rationale explicit and documented, that our ADL can accurately model industrial architectures, and that our SDL is expressive and usable by practitioners.

***Index Terms*—Data-intensive architectures, model-driven engineering, architecture description language, scenario-based design, semi-automated design**

I. Introduction and Motivations

As data increasingly drive decision-making processes, data management becomes a key concern for modern organizations, which need to acquire, persist, analyze, and make data available to internal and external stakeholders based on their needs. Over the years, specialized systems have been designed to manage and process data at scale [1]. Each of them is optimized for specific tasks but no single system meets all organizational requirements [2]. As a consequence, engineers resort to build complex *data-intensive architectures* [3] that integrate multiple such systems to meet the demand of the scenario at hand.

Devising a suitable architecture for a given scenario is challenging, as it requires careful consideration of numerous factors, including input data volume, format, generation frequency, the type of processing to be performed, and the scale of the scenario in terms of number and heterogeneity of data consumers, just to mention a few. To mitigate this problem, several reference architectures have been proposed in the literature, including the well-known Lambda, Kappa, and Liquid architectures [4], all of which inspired our work. However, engineers still need to manually select the most suitable architecture for their application scenario, possibly combining multiple such architectures to fully capture their requirements. Moreover, reference architectures are abstract and high-level, and require additional work to properly configure and adapt them to each specific scenario. In summary, designing a data-intensive architecture for a given application scenario is often a manual and unstructured process, where the software architect's decisions remain implicit and undocumented.

Building on this observation, we adopt a model-driven engineering approach [5] to capture the characteristics and requirements of an application scenario, suggest suitable data-intensive architectures, and guide the selection of concrete software systems for their implementation.

In summary, this paper makes the following contributions. (1) We introduce a *Scenario Description Language (SDL)* to precisely describe application scenarios. Acknowledging the central role of data for architectural concerns [6], the language focuses on data characteristics and data consumers' requirements. (2) We define an *Architecture Description Language (ADL)* to model data-intensive architectures focusing on the functional capabilities of each architectural component. (3) We provide a mapping from architectural components to concrete software systems, based on the functionalities and guarantees they offer. (4) Building on the above artifacts, we propose a novel methodology to automatically generate suitable data-intensive architectures from the description of a given application scenario. We implement such methodology in a prototype interactive tool, which allows software architects to explore the design space and evaluate trade-offs among different architectures under varying operating conditions.

Our approach is motivated by the unique characteristics and challenges of data-intensive architectures. As recognized in recent literature [4], the domain is characterized by a vast design space, which complicates the task of devising a suitable architecture for a given application scenario. The adoption of a model-driven methodology makes design choices explicit and well documented, allowing engineers to rigorously analyze and evaluate how various architectures compare with each other. Instead of committing to a single manually-derived design, the architect is presented with a set of suitable alternatives, and informed about the operating conditions that make each alternative preferable. As acknowledged in [7], in the industrial settings, informal, high-level languages and notations for architectures are often used due to the steep learning curve of formal modeling languages. Moving from this state of things, we provide an SDL that is designed to be simple and intuitive, and we automatically derive more technical ADL-based

architectural models from it. The translation from scenario descriptions to architectures is simplified by a visual and interactive tool, which also highlights the trade-offs among several proposed architectures. Finally, separating the generation of abstract architectures from the choice of concrete systems enables flexibility and portability of the results, allowing our methodology to adapt to different deployment infrastructures and produce solutions that can be migrated across technologies and infrastructure providers.

We assess the correctness of the methodology by applying it to real-world industrial use cases and comparing the generated architectures against expert-designed solutions. We evaluate the quality of the languages by considering their different roles. The SDL is the human-facing input to our methodology, so we focused on evaluating its clarity and usability by involving practitioners in a dedicated survey. The ADL describes automatically generated artifacts, so we primarily judged its expressiveness based on its ability to capture all the concerns that appear in the industrial use cases. The results of our evaluation show that our methodology correctly generates expert-aligned architectures while making the design rationale explicit, that our SDL is expressive and usable by practitioners, and that our ADL can accurately model industrial systems.

The paper is organized as follows. We present our SDL and ADL languages in Section II, and the methodology in Section III. We evaluated our proposal based on the criteria outlined above in Section IV. We survey related work in Section V, and we conclude in Section VI, also indicating future research directions.

II. Modeling Languages

This section presents the SDL (Section II-A) and ADL (Section II-B) languages that underpin our methodology, detailing their core components and relationships using a UML-based notation (Fig. 1)[1]. These modeling languages provide the formal foundation required to rigorously describe application scenarios and their corresponding data-intensive architectures. By making the key characteristics of both the problem (the scenario) and the solution (the architecture) explicit, they enable the automated generation process proposed in this paper.

Each language element has been modeled to capture properties that have a direct and significant impact on architectural concerns in the data-intensive domain. We derived this set of properties from an analysis of reference architectures [8]–[10], their underlying requirements and their structure, identifying the critical factors that differentiate various data management problems and corresponding architectural solutions.

Throughout our presentation, we will use a running example (Fig. 2) based on a real-world use case from CompanyB[2]. The SDL representation of the scenario and the ADL representation of the architecture have been derived from documentation and interviews with CompanyB engineers.

[1]Higher-resolution images at https://anonymous.4open.science/r/ICSA2026.

[2]Names of companies are omitted not to disclose the identity of authors.

A. Scenario Description Language (SDL)

The purpose of the SDL is to capture the requirements of data-intensive applications in a formal yet intuitive way. Accordingly, the SDL follows the typical practice of structuring *data-intensive scenarios*, which involve data being generated by various sources, flowing through multiple transformations, and finally being delivered to a set of consumers [4].

Fig. 1 (top) reports the concepts captured by our SDL and their relations. We represent an SDL *scenario* as a Directed Acyclic Graph (DAG) graph, where *nodes* represent the core data-handling components, and directed *edges* represent the data flowing between them.

Each *Edge moves* data from one node to another, with a specific *period* that defines how often new data is expected. The data itself is represented by a *DataItem*, which is specialized as *Structured*, *Semi-Structured*, or *Unstructured*.

Each *Node* is classified into one of three distinct types, which represent the fundamental roles in the data pipeline. *Producers (P)* represent any component that ingests data into the system. As source nodes of the DAG, they are defined as having one or more output edges but no input edges. They are classified as *Active Producers* if they autonomously deliver data, or *Passive Producers* if they supply data upon request.

In the CompanyB scenario (upper part of Fig. 2), two sources of data exist: a fleet of printers emitting error logs (JSON) and a testing component generating simulation data. This is modeled in the SDL as two active producers that generates semi-structured data: $\texttt{P}_{\texttt{active}}\texttt{1}$ and $\texttt{P}_{\texttt{active}}\texttt{2}$, with millisecond and second period respectively.

Actions (A) are the internal nodes of the graph that represent transformations, each governed by an *input cardinality (ic)* and an *output cardinality (oc)*, which define the amount of data being consumed and produced by the action, respectively. We distinguish between two types of actions based on the origin of the data they process: *Processing Actions* operate on a single data flow (one input edge), while *Merge Actions* integrate two or more data flows (two or more input edges).

The data processing pipeline of CompanyB (upper part of Fig. 2) involves several sequential actions:

- A processing action `a1` enriches the raw printer data from `P1`. It performs a 1-to-1 transformation (`ic`=1, `oc`=1), producing a structured output at an hourly period.
- A merge action `a2` aggregates and structures data from three sources (`P1`, `P2`, and the enriched data from `a1`). It takes 100 input items to produce a single structured output item (`ic`=100, `oc`=1) at an hourly period.
- A processing action `a3` further enriches the data with financial information. This is a high-throughput, 1-to-1 transformation (`ic`=1, `oc`=1) that outputs structured data at a millisecond period.
- A processing action `a4` performs schema alignment and ID resolution on the output of `a3`. It processes data in batches, converting 100 input items into one unified structured record (`ic`=100, `oc`=1) with hourly period.

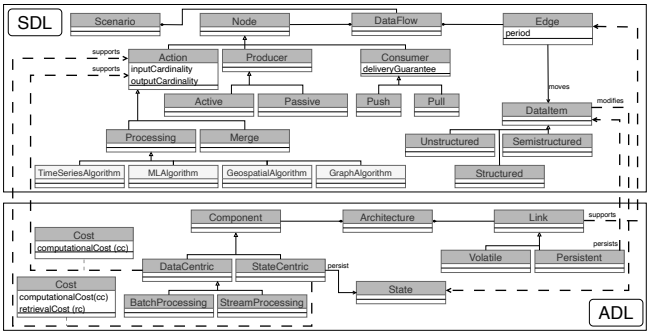


Fig. 1: SDL and ADL languages: components and relations.

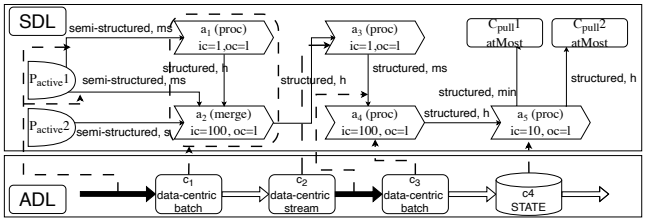


Fig. 2: SDL and ADL representations for CompanyB.

- A processing action `a5` handles final data formatting. It performs a 10-to-1 transformation (`ic`=10, `oc`=1) to convert the data first into Parquet format and then to CSV.

Consumers (C) are the sink nodes that receive the final data products. They represent any entity (e.g., a dashboard) that retrieves data from the system. By definition, they have one or more input edges and no output edges. A critical property is its required *delivery guarantee*, which dictates the semantics of data reception: *at-least-once* ensures no data is lost, but duplicates are possible; *at-most-once* prevents duplicates, but some data loss may occur; *exactly-once* guarantees that each data item is delivered exactly once. We distinguish between two types of consumers: *Pull Consumers* actively query the system to retrieve data on-demand, whereas *Push Consumers* passively receive data delivered by the system.

The CompanyB scenario features two pull consumers that need to query the structured data from `a5` but can tolerate occasional data loss: $\texttt{C}_{\texttt{pull}}\texttt{1}$ and $\texttt{C}_{\texttt{pull}}\texttt{2}$ with a minute and hourly period, respectively.

B. Architecture Description Language (ADL)

Our goal with the ADL is to provide a technology-agnostic vocabulary that is concrete enough to capture the essential architectural decisions without targeting specific software systems. The language formalizes a data-intensive architecture as a graph of software *components* that exchange data through *links* (see lower part of Fig. 1).

Components model classes of software systems that share similar functional roles. We classify them based on the primary architectural distinction between data processing and data storage, a separation of concerns that is central to the design of modern systems [3]. This leads to two main families. *State-Centric Components* are responsible for maintaining an internal, mutable *state*; upon receiving input data, they process it to update their state, which is then made accessible to other components through queries. In contrast, *Data-centric Components* focus on applying data transformations. These are further distinguished into two subtypes: *Stream Processing Components*, which continuously transform data as it arrives, and *Batch Processing Components*, which compute data periodically or on-demand, typically operating on large datasets.

The CompanyB architecture (bottom part of Fig. 2) consists of four components: a batch processing component (`c1`) that validates and processes the initial raw data; a stream processing component (`c2`) that subsequently enriches and transforms the data in real time; a batch processing component (`c3`) that integrates data and produces the unified dataset; and finally, a state-centric component (`c4`) that stores the final results to serve consumer queries.

Links represent the data exchange mechanisms between components. Our language classifies them based on a single property that impacts architectural design: their ability to persist data. Specifically, *Volatile Links* deliver data from a sender to a receiver without storing it. In contrast, *Persistent Links* store the data they transfer, allowing it to be retrieved more than once by receiving components.

In the architecture of CompanyB, data arriving from external sources is transmitted to a cloud-based data lake where it is stored before being processed. This is modeled using a persistent link to ensure the raw data is durably stored upon arrival (see the solid black arrow entering `c1` in Fig. 2). Similarly, the data flowing from the stream component (`c2`) to the subsequent batch component (`c3`) is also stored, a requirement modeled by a persistent link that allows data to be accumulated before batch processing. All other transfers between components use volatile links (white-filled arrows).

A key distinction must be made between persistent links and state-centric components. While both involve storage, their architectural roles and the nature of what they store are fundamentally different. A persistent link serves as a durable buffer for data in transit, such as an immutable log of events or messages. In contrast, a state-centric component processes incoming data to manage and provide a queryable, often mutable, representation of the system's current state.

The core of our model-driven approach lies in the formal relationships between the SDL and the ADL (dashed lines in Fig. 1), which define how application requirements are mapped to architectural solutions. An ADL component *supports* an SDL action if it provides the functional capabilities required to implement that action's data transformation.

In CompanyB (Fig. 2), `c1` supports actions `a1` and `a2`, while `c3` supports `a4`, both enabling batch execution. `c2` supports action `a3` for stream processing. Finally, `c4` supports the data formatting action `a5` and stores the results for querying.

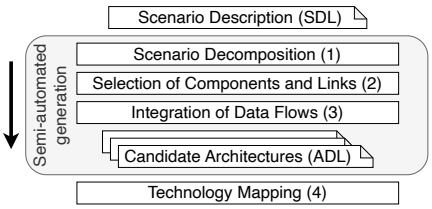


Fig. 3: Overview of the proposed model-driven methodology.

Each component supports an action at a certain *cost*, as the use of different component types has varying implications on the performance of action's execution. We model this cost as an attribute of the supports relationship, represented by the cost association class in our UML model (Fig. 1). We distinguish two cost contributions and assign them to component types: a *Computational Cost*, which applies to all component types and captures the resources required to execute the action, and a *Retrieval Cost*, which applies only to state-centric components and captures the overhead of serving or querying stored results[3].

Precisely defining these cost functions can be challenging, as they often depend on implementation-specific details that are unknown during the early design stages. However, while these costs influence how the methodology generates candidate architectures for a given scenario, in the following sections we show that the methodology is robust to reasonable variations in the cost values: in many cases the choice of architecture is independent of precise costs, and when costs are decisive our approach highlights their impact, bringing the focus of the architect to those critical trade-offs. In addition, we envision a catalog of component profiles that includes precomputed cost estimates for common technologies.

The cost attributes are omitted from CompanyB (Fig. 2) due to unavailable metrics, reflecting a common real-world situation.

Links are mapped to edges in the SDL, as illustrated by the dashed lines in Fig. 1. Specifically, an ADL link *supports* an SDL edge, indicating that it provides the technical means to handle the data flows as required by the scenario.

In CompanyB (Fig. 2) two persistent links support the edges originating from $\texttt{P}_{\texttt{active}}\texttt{1}$ and $\texttt{P}_{\texttt{active}}\texttt{2}$, and one persistent link supports the edge from `a3` to `a4`. All other edges are supported by volatile links, omitted for clarity.

III. Methodology

Having defined the modeling languages that form the foundation of our approach, we now detail the methodology for deriving suitable data-intensive architectures from a given scenario description. The entire process is visually summarized in Fig. 3 and is structured into the following steps:

[3]In general, costs may not be fixed values, but functions that depend on the volume of data they are applied to. For brevity, we use the terms *cost* and *cost function* interchangeably.

1) The input scenario is decomposed into smaller, more manageable sub-problems (Section III-A).
2) An optimization process selects components and links to support each sub-problem (Section III-B).
3) The partial solutions are then merged into a set of candidate architectures, each annotated with the operating conditions under which it is preferable (Section III-C), guiding the choice of the architect.
4) For each selected architecture, concrete data management and processing systems are proposed to implement its components and links (Section III-D).

While the translation itself is automated, the overall methodology is intended to be iterative, with a human expert in the loop. Specifically, an architect can use the generated candidate architectures not only to make a final decision, but also to refine the initial SDL model based on the architectural trade-offs highlighted by the tool, thus supporting an interactive design exploration loop.

In the following subsections, we detail each step of our methodology. The evaluation section (Section IV-B) then provides a complete walkthrough of this process, demonstrating how it is applied to real-world case studies to generate and rank candidate architectures.

A. Scenario Decomposition (1)

As first step, the methodology adopts a divide-and-conquer strategy, transforming the complex, multi-objective problem of satisfying all consumers simultaneously into a set of smaller, independent optimization problems, each targeting the requirements of a single consumer. Accordingly, it decomposes the input scenario's DAG into a set of individual *data flows*. Each data flow is a sub-graph containing all the actions needed to satisfy the requirements of a single consumer, from the producers to that consumer. Nodes and edges shared among multiple paths are replicated in each respective data flow.

B. Selection of Components and Links (2)

After decomposing the scenario in data flows, the methodology starts determining the most cost-effective set of component types (state-centric, batch processing, stream processing) for each specific data flow. We formulate this choice as an optimization problem that associates a cost to each selection of components and aims to minimize the overall cost for each data flow.

Given a data flow DF, we denote the set of its internal nodes as N and the set of its edges as E. Each internal node $n \in N$ defines an action a_n to be performed on its input data. Recall that in a data flow a node can have multiple incoming edges (in the case of a merge node) and multiple outgoing edges. For each edge, we define its *frequency* as the inverse of the *period* specified in the SDL. We denote the incoming frequency f_n^{in} of a node $n \in N$ as the maximum of the frequencies associated to its incoming edges, and the outgoing frequency f_n^{out} of n as the maximum of the frequencies associated to its outgoing edges. Intuitively, the incoming frequency represents the maximum rate at which input data is

requested to upstream components, and the outgoing frequency is the rate at which new results are needed for (requested by) downstream component.

For each node $n \in N$, the optimization problem needs to decide the best type of component to implement n: state-centric, data-centric batch processing, data-centric stream processing. We encode the decision with three Boolean variables x_n^{sc}, x_n^{dc-b}, x_n^{dc-s} for each node $n \in N$, where $x_n^{sc} = 1$ indicates that node n is implemented with a state-centric component, $x_n^{db-b} = 1$ indicates that node n is implemented with a data-centric batch processing component, and $x_n^{db-s} = 1$ indicates that node n is implemented with a data-centric stream processing component. Each node $n \in N$ is implemented by exactly one component, as modeled by the following constraint.

$$\forall_{n \in N} \;\; x_n^{sc} \; + \; x_n^{db-b} \; + \; x_n^{db-s} \; = \; 1$$

We call c_n^{sc} the cost for implementing action a_n of node n using a state-centric component. A state-centric component pays a cost to update its state (driven by input data) and a cost to serve queries (driven by output requests). Its cost is therefore a function of both input and output frequencies:

$$c_n^{sc} \; = \; f_n^{in} \cdot cc(ic_{a_n}) \; + \; f_n^{out} \cdot rc(out_{a_n})$$

where cc is the computational cost, which represents the cost of computing the action and updating the state with the result of the computation, and rc is the retrieval cost to answer requests coming from downstream components. We assume cc to be a function of the input data to be used at each computation, as modeled by the input cardinality attribute ic_{a_n}, and rc to be a function of the output data to be delivered downstream for each request, as modeled by the output cardinality attribute out_{a_n}. Therefore, the cost c_n^{sc} depends on the incoming and outgoing frequencies of n, and on the input and output cardinality ic_{a_n} and out_{a_n}.

We call c_n^{dc-b} the cost for implementing action a_n of node n using a data-centric batch-processing component. This type of components pay a computational cost cc every time the computation is triggered (based on the request of downstream components). Therefore, the cost c_n^{dc-b} depends on the outgoing frequency of n and the input cardinality of a_n, as follows.

$$c_n^{dc-b} \; = \; f_n^{out} \cdot cc(ic_{a_n})$$

We call c_n^{dc-s} the cost for implementing action a_n of node n using a data-centric stream-processing component. This type of components pay a computational cost cc every time a new data item enters the node, which depends on the input cardinality attribute ic_{a_n} for action a_n. Therefore, the cost c_n^{dc-s} depends on the incoming frequency of n and the input cardinality of a_n as follows.

$$c_n^{dc-s} \; = \; f_n^{in} \cdot cc(ic_{a_n})$$

If an action is not supported by a class of components, we assume its cost to be infinite.

The objective of the optimization problem is to minimize the overall cost:

$$\min \sum_{n \in N} \; c_n^{sc} \cdot x_n^{sc} \; + \; c_n^{dc-b} \cdot x_n^{dc-b} \; + \; c_n^{dc-s} \cdot x_n^{dc-s}$$

A key principle of our methodology is to avoid burdening the architect with providing precise cost functions, which are often difficult to estimate. Instead, in the presence of unknown cost parameters, the methodology performs a *sensitivity analysis* by solving the optimization problem multiple times, across a wide range of plausible cost assumptions. This analysis partitions the design space, identifying actions where the optimal component choice is robust (stable across all costs) and those where it represents a critical design trade-off (sensitive to costs). As a result, the outcome is not a single, prescriptive choice for each action, but a set of *candidate component groups*, each containing a specific component choice for every action in the scenario, forming the basis for a *candidate architecture* that is optimal under a particular range of cost assumptions (see Section III-C).

For each candidate component group, the methodology now proceeds to determine the type of link (persistent or volatile) for each SDL edge. This decision is driven by three key architectural principles: ensuring data durability, enabling on-demand queries, and providing data availability for batch processing. To ensure data durability, links from active producers become persistent whenever the consumer requires at-least-once or exactly-once delivery guarantee, enabling durable storage and replay in case of failure. By contrast, links from passive producers (e.g., databases) may remain volatile, as their data is already durable at the source. To support on-demand queries, links to pull consumers become persistent when the upstream component is not state-centric, providing a durable source that non-data-centric components lack. If the upstream component is state-centric, the link remains volatile because its state is directly queryable. Finally, to ensure data availability for batch jobs, an internal link becomes persistent if its downstream component is a batch processor and its upstream component is not state-centric. This allows the batch component to access its complete input dataset by accumulating it in the persistent link, which is required when the upstream source cannot be queried on demand.

C. Integration of Data Flows (3)

For each candidate component group, this step merges the individually optimized data flows to generate a single, coherent candidate architecture. Since a single action n may have been assigned different component types across different data flows, this step applies a set of merging rules to resolve conflicts and select a minimal set of components for the final ADL. Let C^n be the set of component types assigned to action n across all data flows.

The process is guided by two principles: minimizing component duplication and preserving state where necessary. First, to avoid redundancy, if C^n contains both a stream and a

batch component, a single stream processor is selected, as a stream processor can serve both real-time (stream-like) and delayed (batch-like) requests. To support delayed requests, the corresponding output link is made persistent to accumulate results. Second, to preserve queryable state, if C^n contains a state-centric component alongside data-centric ones, both are retained. This reflects a common architectural pattern where a data-centric component performs the computation and a state-centric component stores the results for querying. The methodology models this by creating a single data-centric component (resolving stream/batch conflicts as above) and a state-centric component, connected by a new internal, volatile link responsible for state updates. The link inherits its data type from the original outgoing edges of the data-centric component, and its frequency is set to the maximum frequency among those edges to satisfy the most demanding consumer.

By applying these rules to all actions, the methodology generates a final, unified candidate architecture for each candidate component group, each specifying the minimal set of components and links required to fulfill all consumer requirements.

To guide the architect through the analysis of alternative candidate architectures, the methodology introduces a quantitative *support score* metric. This score is calculated as the ratio of optimal component combinations that result in a specific architecture over the total number of combinations explored. An architecture with a higher score remains optimal across a wider range of cost scenarios. The prototype tool implementing the methodology offers an interactive and visual interface to set cost parameters ranges and assess the implications of their costs. Also, it uses the support score to rank candidate architectures, enabling the architect to make an informed final decision based on quantified trade-offs.

D. Technology Mapping (4)

The final stage of our methodology bridges each candidate architecture with concrete technologies. Indeed, components and links in the ADL represent classes of systems rather than specific technologies, and this step aims to identify suitable options for their implementation.

We intentionally avoid selecting a *single* system for each component. Instead, our methodology relies on categories of systems that share the same core functional characteristics. Indeed, many systems within a given category offer comparable capabilities, and the final choice often depends on factors beyond the scope of our modeling, such as licensing costs, cloud provider constraints, or a team's prior expertise. Thus, our approach guides the architect in selecting from suitable system categories, ensuring the framework remains adaptable as new technologies become available.

Our methodology builds on a taxonomy from recent literature [1]. As part of our contribution, we define the mapping from each ADL element to a corresponding category of systems within that taxonomy, summarized in Table I.

State-centric components map to *data management systems*, whose primary role is data persistence and retrieval. The selection of specific sub-categories (also from [1]) is guided by the structure of the data that modifies its state (see Fig. 1, relations). For structured data items, relational databases are recommended, (e.g., CockroachDB [11], Google Spanner [12]). For semi-structured or unstructured data, NoSQL data management systems are a more suitable choice. This category can be further refined; for instance, document-based data items suggest NoSQL document stores (e.g., MongoDB [13]), while other options include key-value stores (e.g., Redis [14]) or wide-column stores (e.g., Cassandra [15]).

ADL	**System Category** [1]
Batch Processing Component	Data Processing System with Job Deployment
Stream Processing Component	Data Processing System with Task Deployment
State-centric Component	Data Management System
Persistent Link	Data Storage System
Volatile Link	N/A (Communication Channel)

TABLE I: Mapping from ADL elements to system categories.

Data-centric components map to *data processing systems*, which are designed to perform complex computations. The mapping directly follows the specialization defined in the ADL. Stream processing components map to *Data Processing Systems with Task Deployment*, which are designed for continuous, low-latency computation on data in motion (e.g., Apache Flink [16]). Batch processing components map to *Data Processing Systems with Job Deployment*, which aggregate data and process it in large volumes (e.g., Apache Spark [17]).

Finally, links in the ADL map to the technologies used for data transfer between components. Persistent links map to *Data Storage Systems*, which provide durable storage for in-transit data. This includes messaging and queuing systems (e.g., Apache Kafka [18]) or distributed file systems (e.g., HDFS [19]), which enable data persistence between architectural stages. *Volatile links* do not map to a specific system category but rather to a communication channel (e.g., a TCP connection or an API call) that directly connects two systems.

IV. Evaluation

The main goal of our evaluation is to assess the accuracy and practical relevance of our approach. In previous work [20], we already tested the modeling capabilities of our languages and the correctness of the methodology by only applying it to reference architectures from the literature, this evaluation focuses on demonstrating their effectiveness in real-world contexts. Indeed, we aim to show that companies can effectively use our methodology to derive suitable architectures from application scenarios, that our ADL is sufficiently complete to represent real-world data-intensive architectures, and that our SDL is expressive enough to model real-world scenarios while remaining simple and accessible. To this end, we formulate the following research questions:

- **RQ1.** Is our ADL expressive enough to model real-world industrial architectures, and how accurately do the architectures generated by our methodology align with real ones?
- **RQ2.** Is the SDL sufficiently expressive and usable for practitioners to model real-world application scenarios, even without deep expertise in distributed systems?

To answer these questions, we conducted an evaluation using our prototype tool (Section IV-A), which supports visual scenario modeling and the interactive derivation of architectures from them. For RQ1 (Section IV-B), we model real-world industrial architectures using our ADL to assess its completeness, then apply our methodology to the corresponding SDL scenario models, and compare the generated architectures with the real ones to evaluate their correctness. For RQ2 (Section IV-C), we assess the SDL expressiveness and usability through a user study with experts from academia and industry.

To support reproducibility, our tool, all the models discussed in this section, and survey data are fully available[4].

A. Prototype Implementation

We implemented our methodology in a prototype tool that assists architects throughout the design process. The tool is organized into three core modules that mirror the stages of the methodology (Section III), from scenario modeling to the comparative analysis of generated architectures.

Scenario Editor. The first module is a visual editor for modeling application scenarios according to the SDL (Section II-A). It offers a graphical environment to define a scenario as a graph of producers, actions, and consumers, parameterizing each element as shown in the top part of Fig. 1.

Interactive Costs Comparison. This module implements the core analytical logic of our methodology (Section III). It decomposes the SDL scenario into data flows and performs a sensitivity analysis by solving the optimization problem (Section III-B) for each action under different costs assumptions, and allows the user to interactively explore how the optimal component type varies across costs ranges. The user can also fix a specific component type for an action, thus pruning the design space before the final synthesis phase.

Final Architecture Visualizer. This module presents the resulting candidate architectures, which differ only in the component choices for the cost-sensitive actions identified in the previous step. For each action, it considers all suggested component types from the sensitivity analysis. For each combination of these types, it applies the integration rules defined in Section III-C, generating a different architecture. For each such architectures, it also computes and displays its support score. In this way, users can compare architectures and analyze their specific trade-offs before making a final, informed decision.

B. Methodology Evaluation (RQ1)

To answer RQ1, we conducted case studies with two industrial partners: CompanyA, a company specializing in workforce management solutions, and CompanyB, a global technology leader. For each company, we analyzed their real-world data management and processing settings, following a structured three-step process:

[4]https://anonymous.4open.science/r/ICSA2026

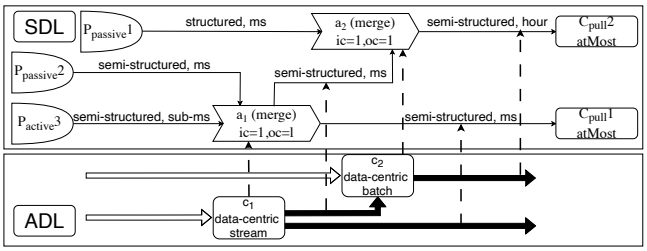


Fig. 4: SDL and ADL representations of the CompanyA scenario and real architecture.

1) **Scenario modeling.** In collaboration with the company's domain experts, we modeled their application scenario using our SDL.
2) **Real architecture.** We documented the company's real architecture, and we show that our ADL can capture all the key characteristics.
3) **Generated architectures and comparative analysis.** We executed our methodology on the SDL model to generate candidate architectures, which we compared against the real architecture to assess our approach's accuracy.

1) CompanyA: Our first case study analyzes the operational data warehouse from CompanyA, which is designed to support fast operational reporting, dashboards, and third-party data integrations. Fig. 4 provides a visual summary of the scenario and the real architecture of the company. Both were derived from a series of interviews with company experts, capturing the functional requirements for the former and the concrete technologies and design choices for the latter.

Scenario Modeling. We begin by capturing the application scenario using our SDL, as in Fig. 4 (top). CompanyA's system ingests data from three distinct sources: an AWS RDS (MySQL) database, an AWS DynamoDB table, and a real-time data stream. The two databases (RDS and DynamoDB) are represented by two passive producers ($\texttt{P}_{\texttt{passive}}\texttt{1}$ and $\texttt{P}_{\texttt{passive}}\texttt{2}$), as they provide data only when queried. Since their update rate is unspecified, we conservatively assume a millisecond period to handle a worst-case scenario for data freshness. Conversely, the real-time stream, which continuously pushes data, is represented by an active producer ($\texttt{P}_{\texttt{active}}\texttt{3}$) with a sub-milliseconds period to represent its high-throughput nature. The RDS producer provides structured data, while the other two provide semi-structured data.

The data processing pipeline consists of two main actions. The first step merges data from DynamoDB ($\texttt{P}_{\texttt{passive}}\texttt{2}$) and the real-time stream ($\texttt{P}_{\texttt{active}}\texttt{3}$) to prepare it for near real-time queries. We model this as a merge action (`a1`) with a 1-to-1 cardinality. This action serves a high-frequency pull consumer ($\texttt{C}_{\texttt{pull}}\texttt{1}$) querying semi-structured data at a millisecond period, thus setting the properties of `a1`'s output edge. This pipeline's output also feeds a second process serving consumers who require hourly updates, represented by a second merge action (`a2`) that combines data from the real-time pipeline (`a1`) and the RDS producer ($\texttt{P}_{\texttt{passive}}\texttt{1}$). This

action has a 1-to-1 cardinality and produces a semi-structured result every hour to serve the final reporting system, which is represented by a pull consumer ($C_{pull}2$).

Real Architecture. CompanyA's implemented solution uses a dual-path approach: a real-time streaming pipeline (AWS Kinesis [21]) for initial enrichment, and an hourly batch process (Apache Spark [17]) for the main transformations. The results are stored in AWS S3 [22] and queried via Presto [23].

The representation of this architecture in our ADL (see Fig. 4, bottom) demonstrates the language's expressiveness. The model captures the system's dual-path nature using distinct stream processing (`c1`) and batch processing (`c2`) components. Crucially, the ADL models the decoupling of storage (S3) and compute (Presto) by employing persistent links, highlighting its ability to distinguish the storage of immutable data in transit from the management of a mutable, queryable state, which would instead be modeled by a state-centric component.

Generated Architectures and Comparative Analysis. We applied the methodology to the CompanyA using the prototype tool in Section IV-A. First, it decomposes the scenario into two distinct data flows, one for each consumer: a high-frequency flow for the real-time consumer ($C_{pull}1$) and a low-frequency flow for the hourly consumer ($C_{pull}2$). Then, for each data flow it identifies the optimal components for its actions. For `a2`, which appears only in the low-frequency data flow, the analysis unambiguously identifies a batch processing component as the optimal choice, regardless of cost assumptions. In contrast, for `a1`, which serves both high-frequency and low-frequency flows, the optimal component choice is sensitive to the relative costs of stream, batch, and state-centric implementations, introducing a design trade-off.

By exploring this trade-off across different cost assumptions, the methodology generates the seven distinct architectures shown in Fig. 5, ranked by their support score to highlight the most stable solutions.

The analysis of the results for the CompanyA case study strongly supports our methodology's accuracy. As highlighed in Fig. 5, the generated architecture with the highest support score (33.3%) consists of a stream processing for action `a1` and a batch processing component for action `a2`, with persistent Links connecting them and leading to the consumers. This design perfectly matches the architecture implemented by CompanyA (Fig. 4). This correspondence is confirmed by our technology mapping (Section III-D), which links ADL elements to system categories from the taxonomy in [1]. Specifically, the generated stream processing component maps to task-deployment systems, a category that includes CompanyA's choice of AWS Kinesis. Similarly, the batch processing component maps to job-deployment systems like Apache Spark, and the persistent link maps to data storage systems like AWS S3, which the company uses as a decoupled storage layer. Furthermore, analysis of the other generated reinforces the value of our approach. The choice of action `a2` (a batch component) remains stable across all solutions, and its implementation with a job-deployment system confirms the methodology's ability to automate robust decisions and focus the architect's effort on the key trade-off, which in this case is implementing action `a1`.

Importantly, the generated candidates represent a systematic exploration of well-known architectural patterns. The highest-scoring solution is a direct instantiation of the Lambda Architecture [9]. Other candidates explore valid alternatives, such as using a state-centric component to model a dedicated Serving Layer Database [24], or using only batch components in a design that mirrors the Kappa Architecture philosophy [8]. This demonstrates that our methodology systematically navigates the design space across different reference architectures, generating coherent designs and providing a quantitative rationale for the critical trade-offs, thus empowering the architect's final decision.

2) CompanyB: Our second case study revisits the CompanyB scenario used as running example in Section II. The corresponding scenario and real architecture are detailed in Section II-A and Section II-B, and shown in Fig. 2.

Generated Architectures and Comparative Analysis. Applying our methodology to the CompanyB scenario, the tool decomposes the SDL model into two data flows, one for each consumer. The subsequent sensitivity analysis reveals that the optimal component choices for actions `a1` (batch), `a3` (stream), and `a4` (batch) are robust across all cost assumptions. In contrast, the choices for `a2` and `a5` are cost-sensitive, introducing design trade-offs. Consequently, the methodology generates a set of candidate architectures that differ in the implementation of `a2` and `a5`. Due to space constraints, we omit the figure with all candidates, but results are reproducible using our tool. Despite presenting multiple architectures, our approach correctly targets the trade-offs that derive from cost sensitivity for `a2` and `a5`, thus focusing the architect's attention on few critical design choices.

The analysis confirms that one of the generated architectures perfectly matches the solution implemented by CompanyB. Interestingly, this architecture is not the one with the highest support score. The most robust candidate (support score of 7.4%) is very similar, differing only in the implementation of action `a5`, using a stream component with a persistent link instead of a state-centric one. This highlights a fundamental architectural trade-off: managing a mutable, queryable state (the company's choice) versus storing immutable data in transit (the proposed alternative).

These results demonstrate the core value of our methodology: it does not prescribe a single *correct* solution, but rather explores the design space, identifies valid alternatives (including the one chosen by experts), and quantifies their relative stability.

C. Expressiveness and Usability of the SDL (RQ2)

To answer RQ2, we conducted an empirical evaluation with participants from both academia and industry. We asked them to model a scenario using our SDL and subsequently to

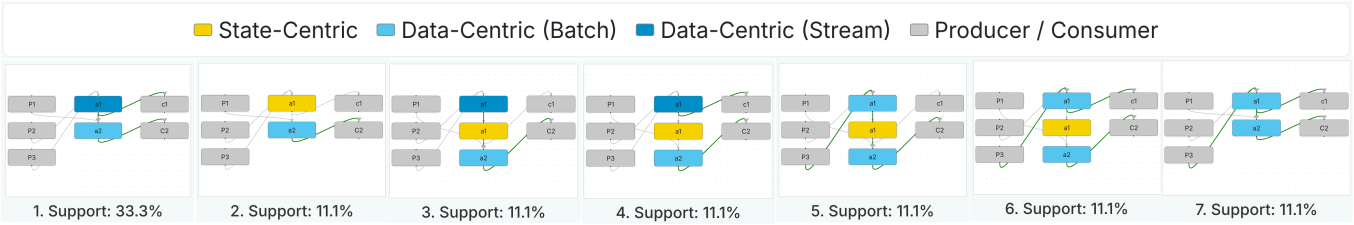


Fig. 5: The seven candidate architectures generated by the methodology for the CompanyA scenario under varying cost assumptions. Representation extracted from the tool *Visualizer* module.

evaluate the language's expressiveness and usability through a structured questionnaire. The goal of the evaluation is twofold: (1) to assess expressiveness, i.e., whether the SDL can capture the architectural drivers of real-world data-intensive scenarios, and (2) to assess usability, i.e., whether practitioners perceive the SDL as understandable, learnable, and suitable for design activities.

The study involved ten participants, including seven industry professionals, comprising data engineers, software architects, and senior developers, and three academic researchers, including faculty members, postdoctoral researchers, and PhD candidates specializing in MDE or distributed systems. This mix provides a balanced perspective between practitioners with daily exposure to data-intensive architectures and researchers with expertise in modeling formalisms. Participants' experience levels ranged from 0-2 years (50%), 3-5 years (20%), to 5+ years (30%), offering insight into both novice and expert perceptions.

1) Expressiveness Evaluation: Participants modeled their application scenarios using the SDL. These scenarios included heterogeneous data sources, multi-stage transformation pipelines, merge operations, consumers with different freshness constraints, and diverse persistence and delivery semantics. Across all exercises, the SDL was able to capture all relevant scenario elements. Table II summarizes the mapping between scenario features and the SDL constructs. Participants emphasized that the SDL's explicit modeling of data characteristics, processing semantics, and flow constraints was particularly valuable during early architectural design, where factors such as frequency, cardinality, and delivery guarantees directly influence whether processing stages should be implemented via batch, stream, or state-centric components. Participants found the SDL effective for modeling real-world data-intensive pipelines, appreciating its representation of data frequencies, cardinalities, and transformation logic. They valued the high level of abstraction and simplicity, which facilitated efficient modeling. While some noted ambiguities in certain symbols and roles, most rarely faced unmodelable concepts.

Overall, feedback indicates that the SDL effectively captures key aspects of data-intensive architectures while allowing for future improvements in notation clarity.

2) Usability Evaluation: Fig. 6 summarizes the usability ratings collected on a 10-point Likert scale. Overall, participants evaluated the SDL very positively across all ten dimensions. They agreed that the SDL captures the core concepts of data-intensive scenarios effectively (7.9/10) and provides adequate support for specifying functional behavior (7.6/10). The sufficiency of constructs received a high score (8/10), indicating that practitioners felt the language offers the right abstractions without redundancy or overcomplexity. Usability-related aspects scored even higher. Ease of learning achieved the highest score (9.4/10), followed by clarity of notation (8.6/10), confirming that the DAG-based structure and explicit modeling of data flow semantics align well with practitioners' mental models. Participants also expressed a strong preference for using the SDL over general-purpose modeling languages such as UML or ArchiMate (8.6/10), citing better alignment with data-intensive architectural concerns. Further, participants rated the SDL as easy to modify (8.7/10), likely to be adopted in real projects (8.1/10), and scalable to larger architectures (7.7/10). Collectively, these results demonstrate that the SDL is not only expressive but also practically usable, satisfying both conceptual and operational requirements of early-stage architectural design.

Scenario feature	SDL construct
Data sources and ingestion patterns	Producer (active / passive)
Data formats (structured, semi-structured, unstructured)	DataItem
Single-input transformations	ProcessingAction
Multi-input transformations	MergeAction
Latency and freshness constraints	Edge period / frequency
Input/output data volume	Cardinality (ic / oc)
Consumer access style	Consumer (pull / push)
Delivery guarantees	DeliveryGuarantee
Stateful vs. stateless processing	Component mapping (data-centric / state-centric)
Persistence requirements	Link (volatile / persistent)

TABLE II: Coverage of scenario features by SDL constructs

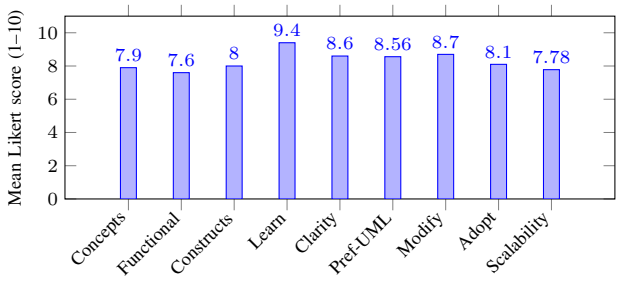


Fig. 6: Mean usability ratings for SDL across different dimensions (1–10 Likert scale).

In addition to subjective feedback, we collected quantitative metrics to assess the modeling efficiency of our approach. Across all participants, the average time required to create

a complete scenario model using our tool was 28 minutes, compared to 75 minutes when using a general-purpose modeling language such as UML or ArchiMate. This represents a 62.7% reduction in modeling time, demonstrating the practical advantages of using a dedicated modeling language and tool tailored to data-intensive architectures. Participants attributed the faster modeling experience to the intuitive DAG structure, domain-specific abstractions, and built-in constraint checking that reduced the need for manual validation.

The results indicate that the SDL is effective for modeling real-world data-intensive scenarios, balancing formal precision with practical usability. This supports RQ2 and highlights the SDL's expressiveness and ease of adoption for practitioners.

D. Threats to Validity

Our evaluation is subject to common limitations of empirical studies on software architecture design. While industrial case studies may not capture the full diversity of data-intensive applications, we mitigate this by selecting two companies with distinct domains, data profiles, and processing requirements; a third case study, focused on an analytical data warehouse, is currently in progress, and we demonstrated that our methodology correctly identifies the trade-offs in reference architectures [20]. Sensitivity to cost assumptions could influence component selection; however, our methodology includes a broad sensitivity analysis to identify stable design choices and expose cost-dependent trade-offs. Survey responses may reflect participant bias or varying expertise, but this risk is reduced through the involvement of both industry practitioners and academic experts with diverse backgrounds.

V. Related Work

Our work spans Model-Driven Engineering (MDE), Requirements Engineering (RE), and automated architecture synthesis. Each area contributes partial solutions, yet none fully addresses the derivation of data-intensive architectures directly from scenario-level requirements.

a) Model-Driven Engineering and ADLs: MDE offers mechanisms for formalizing architectures through metamodels, DSLs, and automated transformations [5], [25], enabling structured refinement from abstract models to concrete implementations [26], [27]. This has led to numerous ADLs addressing structural, behavioral, and variability aspects, including Acme [28], Darwin [29], Wright [30], and Koala [31]. Others, such as SOFA [32], PRISMA [33], and AADL [34], extend support for dynamic evolution and real-time constraints.

However, they all provide limited support for data-intensive architectural drivers, including batch/stream distinctions, delivery guarantees, and persistence semantics, which remain implicit or unsupported. This gap has motivated the need for specialized modeling approaches for data-intensive systems.

b) Domain-Specific Modeling and Reference Architectures: High-level reference architectures such as Lambda [24] and Kappa [35] offer conceptual guidance for mixing batch and stream processing. Yet, they remain informal and require substantial manual adaptation. More formal notations include the DICE UML profile [36], Accordant [37], DATCloud [38], DAF [39], and StreamGen [40]. While these approaches support precise architectural specification or code generation, they presume that an expert has already designed the architecture. Their role is descriptive or generative, not synthesizing architectures from requirements. As such, they are commonly applied for documentation [41] or deployment automation [36] rather than early-stage architectural design.

c) Requirements Engineering Foundations: SDL design draws on RE principles such as goal modeling [42], scenario-based elicitation [43], and the influence of NFRs on architectural structure [44], [45]. Yet, RE methods tailored to data-intensive systems remain scarce [4]. Our SDL therefore explicitly models data characteristics, dataflow semantics, and delivery constraints—features that guide architectural choices but are not captured in typical RE notations. These choices align with the dataflow computation model [46] and established architectural tactics [47].

d) Automated Architecture Synthesis: Automated synthesis has been explored in Search-Based Software Engineering and Goal-Oriented RE [48], [49], typically seeking a single "optimal" architecture using fitness functions [50]. In contrast, our methodology serves as a decision-support mechanism: it identifies trade-offs, highlights cost-sensitive decisions, and generates multiple robust alternatives, consistent with contemporary views of architectural design as a space of competing forces [6], [51]. The cost model used reflects the heterogeneous performance characteristics of modern data systems [2].

e) Technology Mapping: Finally, we leverage established taxonomies of data-intensive technologies [1], [3] to map ADL constructs to concrete categories of systems. This supports architectural solutions that are both principled and directly instantiable with modern data infrastructure.

VI. Conclusions

In this paper, we introduced a model-driven methodology for the systematic design of data-intensive architectures, supported by dedicated modeling languages for describing application scenarios and architectural solutions. The evaluation, conducted on real-world use cases and through expert surveys, demonstrated the practical value of our approach. The methodology generates architectures aligned with expert-designed solutions, and the SDL is both expressive and usable.

The core contribution lies in shifting architectural design from a manual, unstructured to a guided, evidence-based process. By automating the exploration of the design space, our approach brings the architect's attention to the most critical trade-offs, quantifies the stability of different solutions under varying cost assumptions, and makes the design rationale explicit and documented.

As future work, we plan to add a synthesis step to produce deployment artifacts for the generated architectures. This would bridge the gap between high-level design and concrete implementation, leveraging Infrastructure-as-Code [36] and Infrastructure-from-Code [52] techniques to translate architectural models into executable infrastructure definitions.

REFERENCES

[1] A. Margara, G. Cugola, N. Felicioni, and S. Cilloni, "A model and survey of distributed data-intensive systems," *ACM Comp. Sur.*, vol. 56, no. 1, pp. 1–69, 2023.

[2] M. Stonebraker and U. Cetintemel, ""one size fits all": An idea whose time has come and gone," in *Int. Conf. on Data Engineering*, ser. ICDE '05. IEEE, 2005, p. 2–11.

[3] M. Kleppmann, *Designing Data-Intensive Applications*. O'Reilly, 2017.

[4] A. Davoudian and M. Liu, "Big data systems: A software engineering perspective," *ACM Comp. Sur.*, vol. 53, no. 5, pp. 1–39, 2020.

[5] A. R. Da Silva, "Model-driven engineering: A survey supported by the unified conceptual model," *Computer Languages, Systems & Structures*, vol. 43, pp. 139–155, 2015.

[6] N. Ford, M. Richards, P. Sadalage, and Z. Dehghani, *Software Architecture: The Hard Parts*. O'Reilly, 2021.

[7] I. Malavolta, P. Lago, H. Muccini, P. Pelliccione, and A. Tang, "What industry needs from architectural languages: A survey," *IEEE Trans. Softw. Eng.*, vol. 39, no. 6, pp. 869–891, 2013.

[8] J. Lin, "The lambda and the kappa," *IEEE Internet Computing*, vol. 21, no. 5, p. 60–66, 2017.

[9] M. Kiran, P. Murphy, I. Monga, J. Dugan, and S. S. Baveja, "Lambda architecture for cost-effective batch and speed big data processing," in *2015 IEEE International Conference on Big Data (Big Data)*, 2015, pp. 2785–2792.

[10] R. C. Fernandez, P. R. Pietzuch, J. Kreps, N. Narkhede, J. Rao, J. Koshy, D. Lin, C. Riccomini, and G. Wang, "Liquid: Unifying nearline and offline big data integration." in *CIDR*, 2015.

[11] R. Taft, I. Sharif, A. Matei, N. VanBenschoten, J. Lewis, T. Grieger, K. Niemi, A. Woods, A. Birzin, R. Poss *et al.*, "Cockroachdb: The resilient geo-distributed sql database," in *Proceedings of the 2020 ACM SIGMOD international conference on management of data*, 2020, pp. 1493–1509.

[12] J. C. Corbett, J. Dean, M. Epstein, A. Fikes, C. Frost, J. J. Furman, S. Ghemawat, A. Gubarev, C. Heiser, P. Hochschild *et al.*, "Spanner: Google's globally distributed database," *ACM Transactions on Computer Systems (TOCS)*, vol. 31, no. 3, pp. 1–22, 2013.

[13] "Mongodb documentation." [Online]. Available: https://www.mongodb.com/

[14] "Redis documentation." [Online]. Available: https://redis.io/

[15] "Cassandra documentation." [Online]. Available: https://cassandra.apache.org/

[16] P. Carbone, A. Katsifodimos, S. Ewen, V. Markl, S. Haridi, and K. Tzoumas, "Apache flink™: Stream and batch processing in a single engine," *IEEE Data Engineering Bulletin*, vol. 38, no. 4, pp. 28–38, 2015.

[17] M. Zaharia, R. S. Xin, P. Wendell, T. Das, M. Armbrust, A. Dave, X. Meng, J. Rosen, S. Venkataraman, M. J. Franklin, A. Ghodsi, J. Gonzalez, S. Shenker, and I. Stoica, "Apache spark: A unified engine for big data processing," *Communications of the ACM*, vol. 59, no. 11, pp. 56–65, 2016.

[18] "Kafka documentation." [Online]. Available: https://kafka.apache.org/

[19] "Hdfs documentation." [Online]. Available: https://hadoop.apache.org/

[20] Anonymous, "Previous publication from the authors," omitted to preserve anonymity.

[21] "Amazon kinesis." [Online]. Available: https://aws.amazon.com/kinesis/

[22] "Amazon s3." [Online]. Available: https://aws.amazon.com/s3/

[23] "Presto." [Online]. Available: https://prestodb.io/

[24] N. Marz and J. Warren, *Big Data: Principles and best practices of scalable realtime data systems*. Manning Publications Co., 2015.

[25] N. Medvidovic and R. N. Taylor, "A classification and comparison framework for software architecture description languages," *Tran. Soft. Eng.*, vol. 26, no. 1, p. 70–93, 2000.

[26] M. Brambilla, J. Cabot, and M. Wimmer, *Model-driven software engineering in practice*. Morgan & Claypool Publishers, 2012.

[27] S. Kent, "Model driven engineering," in *International Conference on Integrated Formal Methods*. Springer, 2002, pp. 286–298.

[28] D. Garlan, R. T. Monroe, and D. Wile, "Acme: An architecture description interchange language," in *Proceedings of the 1997 Conference of the Centre for Advanced Studies on Collaborative Research (CASCON '97)*, 1997, pp. 169–183.

[29] J. Magee, N. Dulay, S. Eisenbach, and J. Kramer, "Specifying distributed software architectures," in *Proceedings of the 5th European Software Engineering Conference (ESEC '95)*, ser. LNCS 989. Springer-Verlag, 1995, pp. 137–153.

[30] R. J. Allen and D. Garlan, "A formal basis for architectural connection," *ACM Trans. Softw. Eng. Methodol.*, vol. 6, no. 3, pp. 213–249, 1997.

[31] R. van Ommering, F. van der Linden, J. Kramer, and J. Magee, "The koala component model for consumer electronics software," *IEEE Computer*, vol. 33, no. 3, pp. 78–85, 2000.

[32] F. Plásil, D. Balek, and R. Janecek, "Sofa/dcup: Architecture for component trading and dynamic updating," in *Proceedings. Fourth International Conference on Configurable Distributed Systems (Cat. No. 98EX159)*. IEEE, 1998, pp. 43–51.

[33] J. Pérez, I. Ramos, J. Jaén, P. Letelier, and E. Navarro, "Prisma: Towards quality, aspect oriented and dynamic software architectures," in *Third International Conference on Quality Software, 2003. Proceedings.* IEEE, 2003, pp. 59–66.

[34] P. H. Feiler and D. P. Gluch, "Model-based engineering with aadl: An introduction to the sae architecture analysis & design language," *ACM Computing Surveys*, vol. 45, no. 1, pp. 1–37, 2012.

[35] J. Kreps, "Questioning the lambda architecture," O'Reilly Radar Blog, 2014. [Online]. Available: https://www.oreilly.com/radar/questioning-the-lambda-architecture/

[36] M. Artac, T. Borovšak, E. Di Nitto, M. Guerriero, D. Perez-Palacin, and D. A. Tamburri, "Infrastructure-as-code for data-intensive architectures: A model-driven development approach," in *Int. Conf. on Software Architecture*, ser. ICSA '18, 2018, pp. 156–165.

[37] C. Castellanos, C. A. Varela, and D. Correal, "Accordant: A domain specific-model and devops approach for big data analytics architectures," *Journal of Systems and Software*, vol. 172, p. 110869, 2021.

[38] M. Abughazala and H. Muccini, "Datcloud: A model-driven framework for multi-layered data-intensive architectures," in *2025 IEEE 22nd International Conference on Software Architecture Companion (ICSA-C)*. IEEE, 2025, pp. 420–425.

[39] M. Abughazala, H. Muccini, and M. Sharaf, "Architecture description framework for data-intensive applications," in *Int Conf on Intelligent Data Science Technologies and Applications*, ser. IDSTA '23, 2023, pp. 99–106.

[40] M. Guerriero, D. Tamburri, and E. Di Nitto, "Streamgen: Model-driven development of distributed streaming applications," *Tran. Soft. Eng. and Methodology*, vol. 30, pp. 1–30, 01 2021.

[41] M. Ozkaya and C. Kloukinas, "Are we there yet? analyzing architecture description languages for formal analysis, usability, and realizability," in *2013 39th Euromicro Conference on Software Engineering and Advanced Applications*. IEEE, 2013, pp. 177–184.

[42] A. van Lamsweerde, "Goal-oriented requirements engineering: A guided tour," in *Proceedings of the 5th IEEE International Symposium on Requirements Engineering*. IEEE, 2001, pp. 249–262.

[43] B. Nuseibeh and S. Easterbrook, "Requirements engineering: A roadmap," in *Proceedings of the Conference on the Future of Software Engineering (ICSE '00)*. ACM, 2000, pp. 35–46.

[44] M. S. Rahman and H. Reza, "Systematic mapping study of non-functional requirements in big data system," in *2020 IEEE International Conference on Electro Information Technology (EIT)*, 2020.

[45] D. Mairiza, D. Zowghi, and N. Nurmuliani, "An investigation into the notion of non-functional requirements," in *Proceedings of the 2010 ACM Symposium on Applied Computing*, ser. SAC '10. New York, NY, USA: Association for Computing Machinery, 2010, pp. 311–317. [Online]. Available: https://doi.org/10.1145/1774088.1774153

[46] T. Akidau, R. Bradshaw, C. Chambers, S. Chernyak, R. J. Fernández-Moctezuma, R. Lax, S. McVeety, D. Mills, F. Perry, E. Schmidt, and S. Whittle, "The dataflow model: a practical approach to balancing correctness, latency, and cost in massive-scale, unbounded, out-of-order data processing," *Proc. VLDB Endow.*, vol. 8, no. 12, p. 1792–1803, 2015.

[47] G. Márquez, H. Astudillo, and R. Kazman, "Architectural tactics in software architecture: A systematic mapping study," *Journal of Syst. and Softw.*, vol. 197, no. C, 2023.

[48] A. Aleti, B. Buhnova, L. Grunske, A. Koziolek, and I. Meedeniya, "Software architecture optimization methods: A systematic literature review," *IEEE Transactions on Software Engineering*, vol. 39, no. 5, pp. 658–683, 2013.

[49] E. Letier and A. van Lamsweerde, "Reasoning about partial goal satisfaction for requirements and design engineering," *IEEE Transactions on Software Engineering*, vol. 31, no. 11, pp. 938–951, 2005.
[50] M. Harman, Y. Jia, and Y. Zhang, "Achievements, open problems and challenges for search based software engineering," in *2012 34th International Conference on Software Engineering (ICSE)*. IEEE, 2012, pp. 1–12.
[51] M. Richards and N. Ford, *Fundamentals of Software Architecture: An Engineering Approach*. O'Reilly Media, 2020.
[52] I. Aviv, R. Gafni, S. Sherman, B. Aviv, A. Sterkin, and E. Bega, "Infrastructure from code: The next generation of cloud lifecycle automation," *IEEE Software*, vol. 40, no. 1, pp. 42–49, 2023.